

Domain: Electronics

Problem Statement: Design and Implementation of an 8-bit Pipelined Microprocessor in Verilog (Advanced)

Objective

Design and implement an 8-bit pipelined microprocessor using Verilog, incorporating a custom instruction set and basic control flow capabilities. The processor must support instruction-level parallelism through pipelining and should handle both data and control hazards using appropriate techniques such as stalling, forwarding, or dedicated hazard detection logic.

The goal is to not only achieve functional correctness but also ensure that the pipelined architecture correctly preserves data dependencies across instructions. You are free to define the overall architecture, instruction format, pipelining strategy, and hazard-handling mechanisms as part of your implementation.

Instruction Set Requirements

While you are free to define your own instruction set architecture (ISA), your microprocessor must support the following capabilities without encountering pipelining hazards:

- Loading an immediate value into a register
- Performing register-to-register arithmetic operations, including addition, subtraction, and bitwise shifts

Hint: Ensure that both signed and unsigned values are handled appropriately.

- Loading data from memory into registers and storing data from registers into memory
- Executing both conditional and unconditional jumps

Hint: Consider implementing both branch and jump instructions to support flow control.

Evaluation Program

A test program will be provided during the mid-evaluation phase. This program will be used to validate the processor's ability to correctly handle data and control hazards across multiple pipeline stages. The testbench must demonstrate that all instructions execute correctly, maintaining data integrity and timing consistency even under pipelined execution.

Evaluation Criteria

- A complete Vivado project including all Verilog source files must be submitted via a GitHub repository
- The processor must successfully execute the evaluation instruction set using a testbench
- Latency in instruction execution will be considered a key evaluation parameter

Bonus:

- Writing a C program that corresponds to the evaluation instructions, manually compiling it into the custom instruction set, and submitting a detailed document mapping code-level constructs to specific register and memory states within your processor implementation
-

Selection Criterion

- Report specifying basic ideas of architecture to be used , work distribution and proposed project timeline
- The report should clearly mention how the architecture and other aspects can successfully overcome the instruction constraints
- All team members should solve the following problem from HDLBits and share screenshot along with report [7458 - HDLBits](#)

Resources

- [Hardware Modelling using Verilog](#) (For all Verilog based PS)
 - [Lecture -1 Introduction to Computer Architecture](#) (Playlist Videos 1-12)
 - mrce.in/ebooks/Digital Design & Computer Architecture RISC-V Edition.pdf
 - [assembly - Implementation of simple microprocessor using verilog - Stack Overflow](#)
-

Team Size: Maximum 4 members

Problem Statement: Bluetooth-Operated Color Detection System using TCS3200 and UART Communication (Verilog Simulation in Vivado) (Intermediate)

Objective:

Design and implement a Verilog-based system to detect the color of an object using the TCS3200 color sensor. The detected color information will be transmitted via UART, simulating communication with a Bluetooth module. The entire design will be validated through simulation in Vivado.

Tasks Overview:

♦ Task 1: TCS3200 Color Sensor Interfacing

- Interface the TCS3200 color sensor using Verilog.
- Configure the **S2** and **S3** control signals to cycle through Red, Green, and Blue filters.
- Measure the frequency of the sensor's PWM output (**OUT** pin) for each filter and detect the corresponding color.
- The code should run on the following [Evaluation Testbench \(Color Sensor\)](#)

Deliverable: A top-level Verilog module with the following format:

```
module color_detector (           // Module Declaration
input  clk_1MHz, cs_out,         //Inputs : clk_1MHz, cs_out
output reg [1:0] filter, color ); //Output : filter, color
```

♦ Task 2: UART Communication Design

- Design and implement a **UART Transmitter** and **Receiver** in Verilog.
- Transmit the detected color data via UART.
- Receive data (if needed for acknowledgment or control).
- **Deliverables:**

UART Receiver Top Module Format:

```
module uart_rx(  
  
    input clk_3125,  
  
    input rx,  
  
    output reg [7:0] rx_msg,  
  
    output reg rx_parity,  
  
    output reg rx_complete);
```

[Uart RX Evaluation TestBench](#)

UART Transmitter Top Module Format:

```
module uart_tx(  
  
    input clk_3125,  
  
    input parity_type,tx_start,  
  
    input [7:0] data,  
  
    output reg tx, tx_done );
```

[Uart TX Evaluation TestBench](#)

♦ Main Task: System Integration and Simulation

- Integrate the TCS3200 color detection module with the UART transmitter and receiver.
- Design a comprehensive testbench to simulate continuous color detection and UART data transmission.
- Validate the integrated system using Vivado simulation.

Submission Requirements:

- A zipped folder containing:
 - Complete Vivado project.
 - All Verilog source files (TCS3200 interface, UART modules, top-level integration).

- Simulated testbench waveforms demonstrating successful color detection and UART communication.
-

Evaluation Criterion :

- Successfully Running the Uart Rx , Uart Tx and Color Sensor Testbenches
 - The number of logic elements used should be as minimum as possible
-

Important References:

- [Hardware Modelling using Verilog](#) (For all Verilog based PS)
 - [TCS3200 Color Sensor Documentation](#)
 - [Universal Asynchronous Receiver Transmitter \(UART\) - Google Docs](#)
 - [Arduino Color Sensing Tutorial - TCS230 TCS3200 Color Sensor](#)
 - [Universal Asynchronous Receiver-Transmitter \(UART\)|Verilog implemented code with simulation results](#)
-

Selection Criterion

- All team members should solve the following problem from HDLBits and share screenshot along with report [7458 - HDLBits](#)
 - Basic Report with plan to integrate color sensor with UART
-

Team Size: Maximum 4 members

Problem Statement: Design and Implementation of a hardware accelerator for quaternion multiplication and use it to calculate real-time orientation using IMU data

(Advanced)

Overview:

Quaternion representation of orientation is a widely used concept in robotics, graphics, aerospace, AR/VR, etc., because of its efficiency and ease in computation. One of the features that has to be implemented while computing orientation is quaternion multiplication, which becomes a bottleneck in some real-time use cases because of its slight complexity. So the aim of this project is to make an efficient processor for processing quaternions from IMU data.

Description:

- The main objective of this project is to offload cpu for quaternion multiplication during the processing of IMU data.
 - Write Arduino Code to extract data from MPU6050 IMU through a proper communication protocol (I2C, SPI, etc.,).
 - Implement modules to get data from the Arduino for offloading the main processor.
 - Then create hardware blocks for implementing quaternion multiplication through direct formulas, and then accelerate this process using specific algorithms (like Hadamard matrices). Implement both blocks on the PYNQ Z2 board.
 - Then make a pipelined version for both direct and algorithmic multiplication and perform the implementation again. Compare and report them
 - We will give a file of IMU data. Use that data to testbench your designs and compare the results.
-

Specifications:

- The incoming data from the IMU is a 16 bit signed integer. You should extract that data and preprocess it (such as filtering) and send the angular velocity data to fpga which processes the quaternion data
- Choose a suitable communication to transfer data from arduino to FPGA and create a data buffer to avoid sample rate issues.

- Convert the angular velocity data to quaternions and store the data in data memory. The data memory should include the memory of previous orientation quaternion which should be used to calculate new orientation.
 - Perform the Quaternion multiplication using your proposed algorithm and store it in memory.
 - Again create an output data buffer and send it to arduino which should display the quaternion data in its serial monitor.
 - For simulation purposes you can directly store and get data from a file of IMU data and represent the output data during the test bench in Vivado itself. During hardware implementation the output should be on the serial monitor of the arduino.
-

Resources:

Basic Digital Electronics:

<https://youtube.com/playlist?list=PLBlnK6fEyqRjMH3mWf6kwqiTbT798eAOm&feature=shared>

Verilog:  Introduction

Quaternions:  Quaternions | Robotic Systems

 Quaternions

IMU:

<https://cdn.sparkfun.com/datasheets/Sensors/Accelerometers/RM-MPU-6000A.pdf>

Deliverables and Documentation:

- Arduino Files
 - Verilog Files, along with constraint files, used for implementation
 - A video and screenshots of simulations done
 - The output files.
 - A report on comparisons and insights from the results
-

Evaluation Criteria:

The evaluation is based on

- Performance of the design
- Number of hardware resources used
- Efficiency in the algorithm
- Overall time taken to process the quaternion

BONUS: Implementation of I2C Master logic as a module and connecting it to the accelerator block gives you extra points during evaluation.

Selection Criterion

- All team members should solve the following problem from HDLBits and share screenshot along with report [7458 - HDLBits](#)
 - A proposal according to the IITISOC format
-

Team Size: Maximum 4 members

Problem Statement: Design and Simulation of a Multi-Rail Computer Power Supply System with Feedback Control

(Intermediate)

Objective:

Design and simulate a regulated multi-output DC power supply for a computer in MATLAB Simulink. The system should take a standard AC input and generate stable DC rails using power electronics and feedback control.

Core Components (Required Scope):

1. **AC to DC Front-End (Simulated in Simulink)**
 - Full-bridge rectifier with LC filtering
 - Input: 230V AC, 50Hz → Output: ~310V DC (unregulated)
 - Switching system
 - i. Charge battery from mains when available
 - ii. Automatically switch laptop load to mains supply once battery is full
 - iii. In case if mains is disconnected, immediately switch the load to the battery without interruption
 2. **DC-DC Conversion Stage**
 - Buck converters generating +12V, +5V, and +3.3V rails
 - PWM switching with closed-loop control using PI or PID feedback
 3. **Control & Monitoring**
 - Real-time voltage tracking via scopes or dashboard elements
 - Fault indicator blocks for over/under-voltage detection (simulation only)
-

Bonus:

1. **EMI-Aware Multilayer PCB Design (EasyEDA/EagleCAD)**
 - Schematic and layout of the simulated system
 - Stack-up planning with power/ground planes
 - EMI mitigation strategies: decoupling, star grounding, routing rules
 2. **Parasitic and Signal Integrity Modeling**
 - Include ESR of capacitors, trace inductance, and switching parasitics in simulation
 - Observe impact on voltage and current ripple
-

Submission Requirements:

1. Complete Simulink simulation file with load testing done on RL Loads.

2. Waveforms pictures for every peripheral used (peripherals to be modeled using RL or RLC Loads)

Evaluation Criteria:

1. Low ripple content in voltage and current waveform is suitable for every peripheral that uses a DC input.
 2. Efficiency of DC/DC Converter will also be assessed.
 3. Overall circuit design, bulkiness, size, will also be considered.
-

Resources:

1. [MIT OpenCourseWare for PE](#) (Watch till DC/DC converters)
2. [Closed Loop Control for Buck converter](#)

- **Selection Criterion**

A proposal according to the IITISOC format. Proposal should -:

- Clearly mention the overall details of converters to be used in the chain
 - Resources from Matlab Simulink to be used in the process
 - Address the control aspects of the converters with brief description of your ideas
-

Team Size: Maximum 4 Members

Project Title: Design of a Memory-Constrained Path Planner in C for RISC-V based Microcontroller (Intermediate)

Objective-:

In embedded systems and robotics, efficient use of limited hardware resources is critical. This project focuses on designing a compact navigation software stack for a hypothetical autonomous robot navigating a 16×16 grid environment with static obstacles. The software is developed in pure C, targeting the RV32I RISC-V instruction set, and is simulated using the RIPES processor simulator.

The robot must calculate the shortest path between a start and an end point, both of which are received via a memory-mapped UART interface. The environment is predefined and stored as a binary occupancy grid, with each cell indicating the presence or absence of an obstacle.

Upon receiving the coordinate input via UART, the robot's control logic must compute a path using a memory-efficient algorithm, compress the resulting path, and simulate movement along this path. A virtual IR Sensor (memory-mapped) provides feedback about whether the robot remains aligned with the path.

The software must execute under strict memory and instruction constraints

System Overview

Inputs:

- UART (memory-mapped): Sends start and goal coordinates in a fixed format.
- Static grid map: 16×16, stored in bit-compressed form.
- Sensor input: Simulated via a memory-mapped register indicates alignment with path.

Functionalities:

1. UART Interface:

- For receiving start and end coordinates

2. Grid Initialization:

- Predefined grid, statically stored in program memory or initialized in RAM.

3. Pathfinding Algorithm:

- BFS, Dijkstra, A-Star or any other such algorithm optimized for minimal memory.
- Uses flat arrays or direction maps, no dynamic memory.
- Produces a list of movement directions (e.g., UP, DOWN, LEFT, RIGHT).

4. Path Compression:

- Path stored in using bit encoding per direction, packed into `uint8_t` array.
- Ensures full path fits within limited memory.

5. Simulated Movement:

- Robot follows the path, step-by-step.
- On each step, read the sensor register to check alignment.
- If misaligned, enter a correction or wait state.

6. Main FSM:

- Handles the overall state

Constraints

- Instruction memory: ≤ 512 instructions (RV32I)
 - Data memory: ≤ 64 words = 256 bytes
 - No dynamic memory (i.e., no malloc, no recursion)
 - Entire codebase written in C, compiled with `riscv32-unknown-elf-gcc`
-

Evaluation Strategy

The entire solution will be evaluated using the RIPES simulator, and the following criteria will be used:

1. Correctness
 2. Instruction Count
 3. Data Memory Usage
 4. Code Quality
 5. Sensor Logic
 6. Simulation Demonstration in RIPES
-

Resources

[Path Planner - Google Docs](#)

[Bitwise Operators in C \(Part 1\)](#)

[RISC-V Assembly Code #1: Course Intro, Registers](#)

Selection Criterion

A proposal according to the IITISOC format. Proposal should -:

- Share approach of the algorithms to be used for path planning that are being considered
 - Brief Description of Ideas used for optimizing the challenges mentioned in the PS
-

Team Size: Maximum 4 Members

Problem Statement: AmpaNova (Intermediate)

Objective:

Simulate a **mixed-signal function generator** with integrated **analog computing**, waveform shaping, and modulation capabilities, all integrated in a single device (file). Circuit Designing must be done either in a Spice Software (LTSpice/Pspice) or in MATLAB-Simulink.

Core Requirements:

1. Waveform Generation:

Generate **sine, square, triangular, quadratic** waveforms using analog blocks, like opamps, basic RLC oscillator circuits etc and using simple multiplier circuit to multiply 2 waveshapes

2. Analog DE Solver:

Solve **differential equations** using op-amp-based circuits. Input to the solver can be any waveform or shaped signal. And the output will now be your new waveform of the function generator. The DE should have tunable parameters setted using potentiometers in real time.

$$a_1 \frac{d}{dt} y + a_0 y = f(x)$$
$$a_2 \frac{d^2}{dt^2} y + a_1 \frac{d}{dt} y + a_0 y = f(x)$$

3. Wave Shaping:

Use analog shaping circuits (clippers, log/exp, rectifiers and many more) and allow shaped signals to be routed to output of FG or into DE solver to generate more waveshapes.

Bonus:

1. Digital Input + DAC:

Accept digital waveform input, convert via DAC, and process like other signals.

2. Storing into a memory:

Store one period of any waveform using an ADC and reproduce it cyclically in analog form if required.

3. Switchable Control:

Entire system must switch between modes via **analog MOSFET control** only (no microcontrollers). And reduce the number of components used.

4. **Filtering:**

Implement filters to extract specific harmonics from any input, and let the user define the frequency (maybe by using potentiometers)

Submission Requirements:

1. Simulation File, either Spice or MATLAB-Simulink is acceptable
 2. A document, listing your approach, problems encountered, solutions, calculations (if required, not necessary)
 3. Screenshots of output waveforms of all the functionality incorporated with proper labelling of the pictures
-

Evaluation Criteria:

1. Well-organized circuit layout, minimalist design, waveform accuracy will be judged
 2. In case of any non-ideal behaviour, mention it in the document
 3. Waveforms must include:
 - a. Sine wave, triangular wave, square wave generation circuit output
 - b. Differential solver output waveform with input as clipped and clamped sine, triangle and quadratic wave
 - c. Multiplication of two waves with different frequencies
-

Resources:

1. [Intro to Laplace Transform and Transfer function](#)
2. [Circuit analysis using laplace transforms](#)
3. [Wave Shaping circuit \(Clipper\)](#)
4. [Wave Shaping circuit \(Clamper\)](#)

● **Selection Criterion**

A proposal according to the IITISOC format. Proposal should -:

- Highlight the various overview of circuit blocks to be used for several functionality
 - Optimization ideas within the circuit blocks for the PS to be done with minimal resources
-

Team Size: Maximum 4 Members

Problem Statement: Smart Automated Parking and Safety Management System for Multilevel Parking in IIT Indore (Beginner)

Overview

Considering the ongoing work of multilevel parking in IIT Indore , efficient utilization of parking space has become a necessity. Traditional parking lots often lead to inefficiencies, vehicle collisions, and a lack of real-time monitoring. To address these issues, a **Smart Parking and Safety Management System** must be designed using the **Arduino platform**, combining automation, safety features, and real-time status indication.

Objective

Design and implement an embedded system-based Smart Parking System that automates the entry and exit process using servo-controlled barriers, indicates real-time slot availability through visual indicators, and ensures safety using proximity-based alerts. The system should be scalable, modular, and simulate an intelligent parking lot management solution for future smart cities.

Functional Requirements

1. **Automated Entry/Exit Mechanism:**
A servo motor-controlled gate must open when a vacant parking slot is detected and a car is at the entrance (using IR or ultrasonic sensors). The gate must automatically close after the car enters.
2. **Slot Detection and Indication System:**
Each parking slot must be equipped with a sensor (IR or ultrasonic) to detect vehicle presence. Corresponding **RGB LEDs** must indicate this
3. **Safety and Anti-Collision System:**
4. **Overstay Monitoring:**
5. **Real-Time Slot Display Module:**
6. **Night Mode and Lighting Control:**
7. **Manual Override for Security:**
8. **Energy-Efficient Design:**
(**Bonus** : Considering use of Solar Panels to automate all the electrical lighting and mechanism used in the parking system)

Constraints

- The system must be developed using any microcontroller (by justifying its use) with Arduino IDE platform using C/C++.
 - Minimum of **4 parking slots** in prototype .
 - Use commonly available beginner-level components (e.g., IR sensors, ultrasonic sensors, LEDs, servo motors, buzzer, LCD/7-segment display).
-

Deliverables

1. **Technical Report (PDF)** – containing:
 - Problem analysis
 - System architecture and component selection
 - Circuit diagrams and wiring schematics
 - Flowcharts/state diagrams of software
 - Safety analysis and error-handling strategy
 - Cost estimation and scalability discussion
 2. **Source Code** – well-commented source files for:
 - Gate control
 - Slot monitoring and indication
 - Collision detection
 - Lighting and display module
 - Bonus feature implementations (if any)
 3. **Simulation Demonstration** – showing:
 - All features functioning in a working model
 - Explanation of key modules and innovative features
-

Evaluation Criterion

- Optimal use of resources , along with handling of all cases and complete functionality
 - Source code clarity
 - Bonus Solar implementation
-

Selection Criterion

A proposal according to the IITISOC format. Proposal should -:

- Highlight the various components used and action plan along
 - Timeline for further aspects along with work distribution
-

Team Size: Maximum 3 Members

Problem Statement: Smart EV Shuttle Tracking and Booking System for IIT Indore (Beginner)

IIT Indore's sprawling campus has adopted electric shuttle buses to promote sustainable transportation. However, the current system lacks digital infrastructure for real-time location tracking, shuttle availability updates, and passenger interaction. Students and staff often wait at stops without any visibility into shuttle positions, leading to inefficiencies, uncertainty, and dissatisfaction.

This project proposes a **collaborative, cross-disciplinary solution** that leverages **embedded systems and mobile application development** to create a **smart campus EV shuttle tracking and booking system** tailored for IIT Indore.

Project Objective

To design and implement an integrated system for **real-time shuttle location tracking, occupancy monitoring, user booking, and route visibility** through:

1. An **embedded onboard unit** installed in each EV shuttle.
2. A **cross-platform mobile application** for students and staff.

The system should improve user experience, streamline shuttle usage, and serve as a model smart transportation solution for educational campuses.

System Components

A. Embedded Hardware System (Electronics Team)

Each shuttle will be equipped with a smart embedded unit built around an ESP32 microcontroller. Key responsibilities include:

1. **GPS Tracking**
2. **Data Transmission with Store-and-Forward**
 - Due to unreliable Wi-Fi connectivity on moving vehicles, data will be **locally stored** and **automatically uploaded** when the shuttle is within range of known **campus Wi-Fi hotspots** (e.g., terminal stops).
 - Demonstrates **delayed data synchronization** and **resilience to connection loss**.

3. **Occupancy Detection:**

- Install IR or ultrasonic sensors at doors to estimate number of passengers onboard.

4. **Local Display Interface:**

- Onboard OLED/LCD screen shows route ID, current occupancy, GPS lock status, and alerts.

This system teaches practical skills in GPS interfacing, wireless communication, sensor integration, power management, and fault-tolerant IoT design.

B. Mobile App and Cloud Integration (Software Team)

Develop a lightweight, cross-platform mobile app using **Flutter** or **React Native**. Key features:

1. **Map-Based Live Tracking:**

- Visual display of shuttles on a campus map, showing route ID and estimated arrival time at stops (based on latest data received from ESP32 units).

2. **Occupancy Awareness:**

- Show live occupancy level (e.g., Low, Medium, Full) to help users plan accordingly.

3. **Pickup and Drop Selection:**

- Users select desired boarding and destination stops to receive ETA and route match.

4. **Push Notifications:**

- Alerts for approaching shuttle, emergency status, or system messages.

5. **Data Backend:**

- Firebase or REST API backend to store shuttle data, occupancy, and trip logs.

6. **Admin/Driver App or Web Panel (Optional):**

- Interface for route updates, driver check-in/out, or analytics dashboard.

This system builds real-world knowledge in full-stack development, API communication, map rendering, and user-focused mobile UX design.

Deliverables

Embedded Systems Team:

- Fully wired simulation with necessary components.
- Source code with proper documentation and comments.
- PCB layout of the schematic.
- Final technical report (architecture, flowcharts, testing results).

App Development Team:

- Cross-platform mobile app with map, occupancy, booking features.
 - Source code with setup instructions.
 - Firebase/REST API backend.
 - Final report (app flow, backend structure, UI/UX rationale).
 - APK or installation link.
 - Screenshots and demo video.
-

Evaluation Parameters

- Optimal use of resources that ensures complete working of the system.
- GPS integration and data handling under all circumstances.
- Flutter/React Native development.
- Real-time databases and map rendering.

Selection Criterion

A proposal according to the IITISOC format. Proposal should -:

- Highlight the various components used and action plan along
- Timeline for further aspects along with work distribution

Team Size: Maximum 3 Members